



BITS Pilani

Cloud Computing

SEWP ZG527

Agenda

From Virtualization Fundamentals to IaaS Architecture



01



VIRTUALIZATION RECAP

Refresh the foundations
Virtual Machines · Hypervisors ·
Resource Abstraction

02



HYPERVISORS & VM ARCHITECTURE UNDERSTAND

- What is a Hypervisor?
- Hypervisor Types (Type 1 vs Type 2)
- Hypervisor Techniques
- Resource Sharing in VM's
- Compare & Contrast architectures

03



IAAS – DEEP DIVE

- What is IaaS – Recap
- IaaS Architecture & Reference Model
- Amazon Web Services (AWS) –
An IaaS Reference
- Regions, AZ's & Edge Locations

01

FOUNDATION

Understand the basics

02

ABSTRACTION

Deep dive into the hypervisor layer

03

CLOUD

From virtualization to cloud scale



By the end of this session, you'll connect virtualization concepts to real-world cloud infrastructure.





BITS Pilani

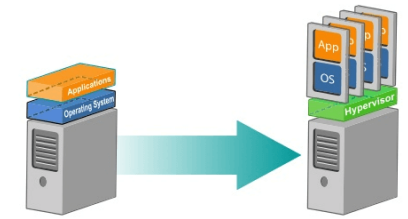
Pilani | Dubai | Goa | Hyderabad



Recap



What is Virtualization?



What is Virtualization?

One Physical Machine (Host). Multiple Virtual Worlds (Guests).



Virtualization is a computer architecture technology that allows **multiple virtual machines (VMs)** to be multiplexed on the same physical hardware (**Host**).



Run Multiple Operating Systems (Guests)
Virtualization enables multiple operating system instances to run **concurrently on a single computer (Host)**, isolated from each other.

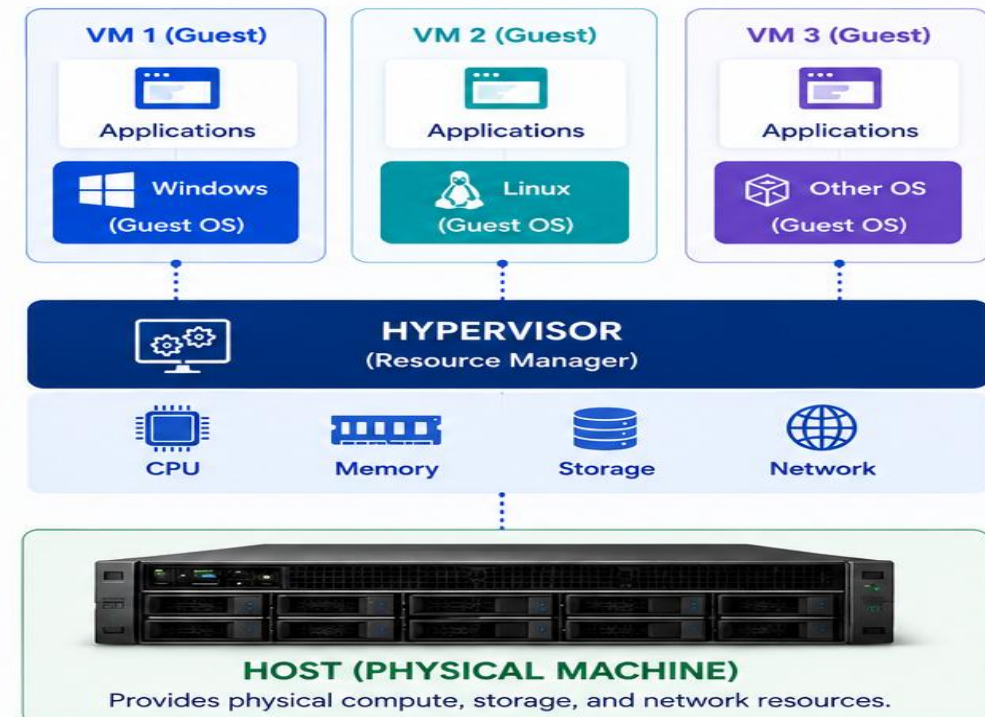


Maximize Hardware Efficiency (Host)
Instead of buying and maintaining separate hardware for each application, each one gets its own operating system while **sharing the same physical resources**.



Intelligent Resource Management (Host)
The hypervisor (or operator) manages and allocates CPU, memory, storage, and other resources to each VM (Guest), ensuring **every guest gets what it needs**—nothing more, nothing less.

MULTIPLE VIRTUAL MACHINES (GUESTS)



Virtualization abstracts hardware to create isolated, secure, and efficient environments, enabling better utilization, agility, scalability, and cost savings.



Better Utilization



Lower Costs



High Availability



Scalability & Agility

New Terms to Remember

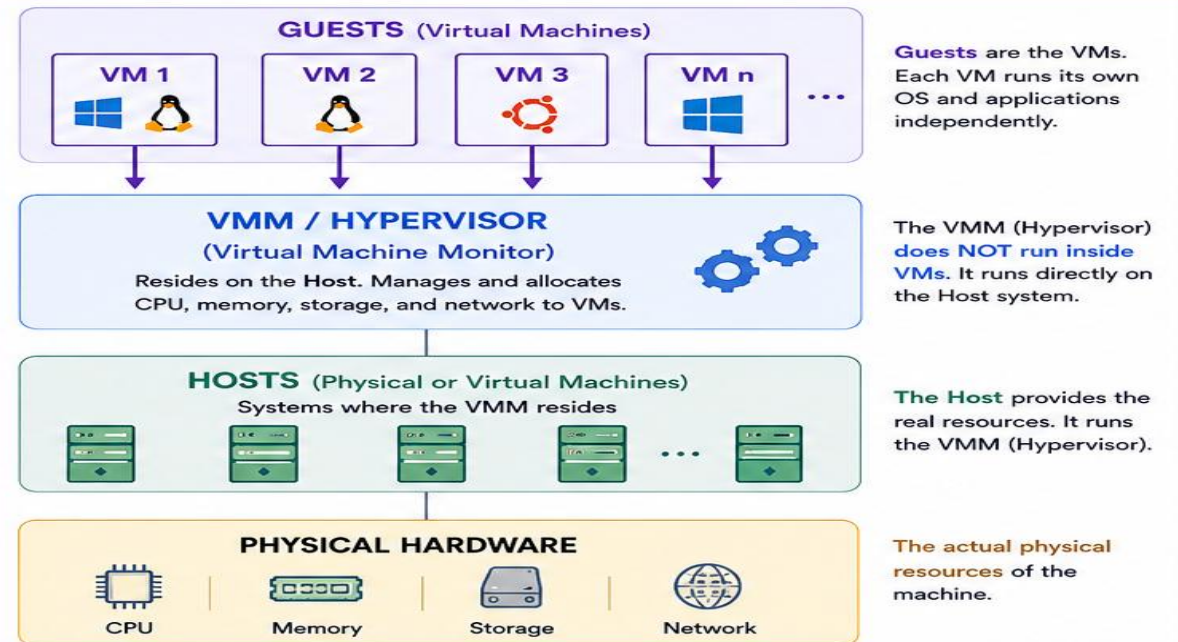
KEY TERMS IN VIRTUALIZATION

Remember all 7 key terms with this complete mnemonic and visual!

MNEMONIC: “**MY** **VERY** **HAPPY** **MANY** **HOSTS** **RUN** **GUESTS**”

M	My	→	VM Virtual Machine	A virtual computer created to run an OS and applications.	
V	Very	→	VMM Virtual Machine Monitor	The software layer that manages and controls VMs.	
H	Happy	→	Hypervisor = VMM Another name for the Virtual Machine Monitor	It creates, runs and manages VMs and allocates resources.	
M	Many	→	Multiplexed Many or several	The VMM multiplexes (shared use of) the hardware among many VMs.	
H	Hosts	→	Host System where the VMM resides	The physical (or virtual) machine that runs the hypervisor.	
R	Run	→	Runs Supports and executes	The host runs the VMM, which runs the VMs and their workloads.	
G	Guests	→	Guest Virtual Machines created	The VMs are called guests because they run on the host.	

THE BIG PICTURE: WHERE EVERYTHING LIVES



QUICK MEMORY TIPS



A VM is a virtual computer inside the system.



VMM (Hypervisor) is the manager of all VMs.



Multiplexed means sharing one host's resources among many VMs.



Hosts are the real systems that host the VMM.



Runs means the host runs the VMM, which runs the VMs.



Guests are the VMs that run on the host, managed by the VMM.



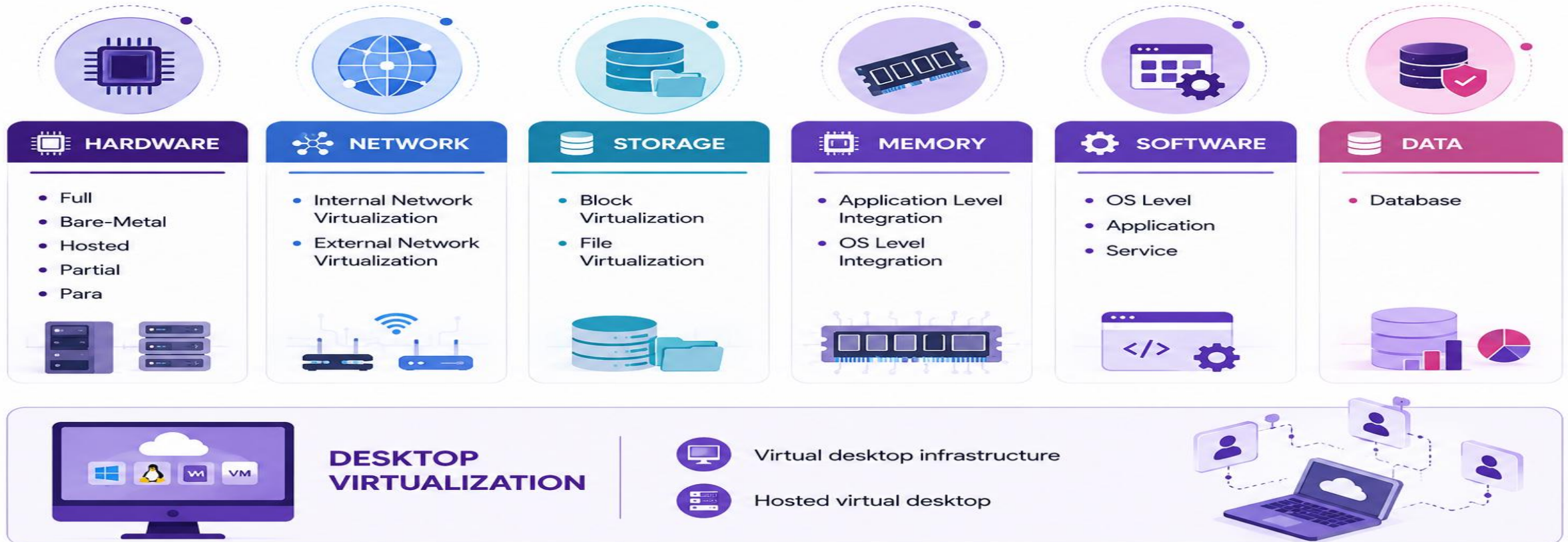
Hypervisor and VMM are the same thing!

★ Think: **My Very Happy Many Hosts Run Guests** — remember all 7 key terms!

Where do we see Virtualization

VIRTUALIZATION

Virtualization abstracts resources to run multiple workloads efficiently, securely, and independently.

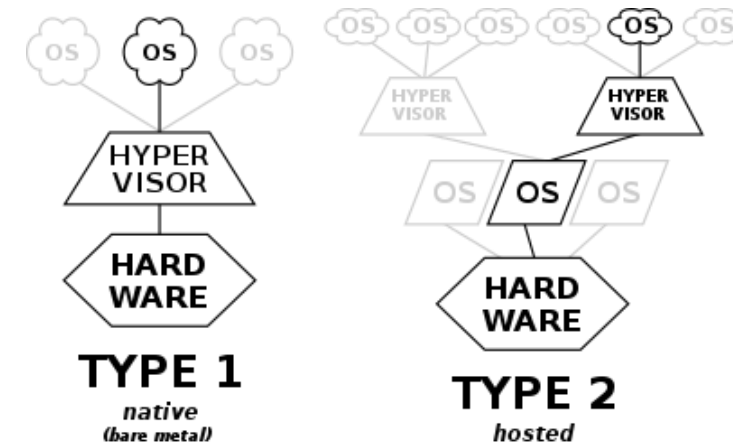




BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Understanding Hypervisor



Virtualization Stack

Virtualization can be implemented at **different layers** of the **computing stack**, with each layer providing **abstraction, isolation, portability, and better resource utilization**.

Application Level – virtualizes applications and their runtime environments.

Library/API Level – provides compatibility by virtualizing system libraries and APIs.

Operating System Level – isolates multiple user environments on a shared OS kernel.

Hardware Abstraction Level – virtualizes CPU, memory, storage, and I/O to create VMs.

Instruction Set Level – translates instructions so software built for one CPU architecture can run on another.

LEVELS OF VIRTUALIZATION

From Applications to Hardware

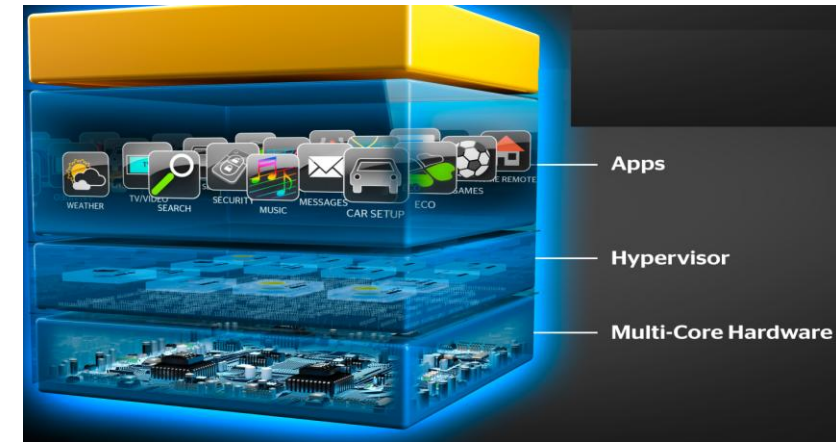
LEVEL	WHAT IT VIRTUALIZES	EXAMPLES / TECHNOLOGIES	VIRTUALIZES
5 Application Level	 Virtualizes applications so they can run in isolated environments without changing the underlying operating system.	JVM / .NET CLR / Panot	 Applications
4 Library (API) Level	 Virtualizes system libraries and APIs to provide portability and compatibility across different OS environments.	WINE / WABI / LxRun / Visual MainWin / vCUDA	 API / Libraries
3 Operating System Level	 Virtualizes the operating system kernel to create isolated user spaces and multiple virtual environments.	Jail / Virtual Environment / Ensim's VPS / FVM	 Operating Systems
2 Hardware Abstraction Layer (HAL) Level	 Virtualizes hardware resources (CPU, memory, storage, I/O) by providing an abstraction layer between OS and hardware.	VMware / Virtual PC / Denali / Xen / L4 / Plex 86 / User mode Linux / Cooperative Linux	 Hardware
1 Instruction Set Architecture (ISA) Level	 Virtualizes the instruction set so that binary code compiled for one architecture can run on a different architecture.	Bochs / Crusoe / QEMU / BIRD / Dynamo	 Instruction Sets (ISA)



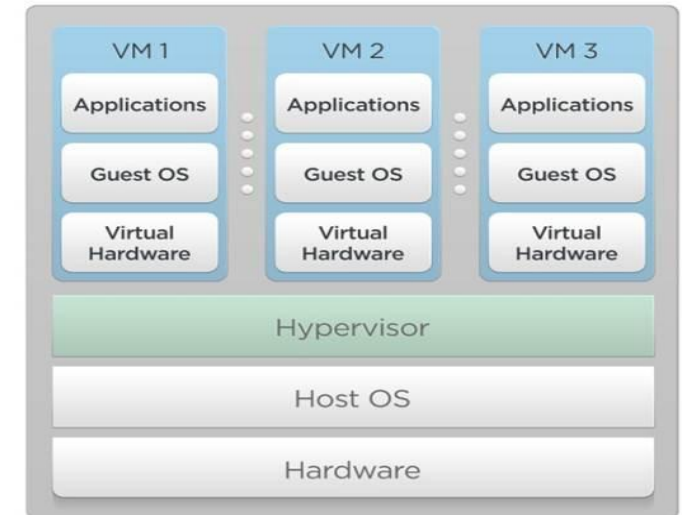
Virtualization exists at multiple levels of the computing stack. Each level provides a different abstraction, enabling flexibility, isolation, portability, and efficient resource utilization.

Hypervisor

- A **hypervisor** or **virtual machine monitor (VMM)** is computer software, firmware, or hardware. VMM creates and runs virtual machines.
- A computer on which a hypervisor runs one or more virtual machines is called a host machine,
- Each virtual machine is called a guest machine
- The hypervisor presents the guest systems with a virtual operating platform and manages the execution of the guest operating systems.
- Multiple instances of a variety of operating systems may share the virtualized hardware resources:



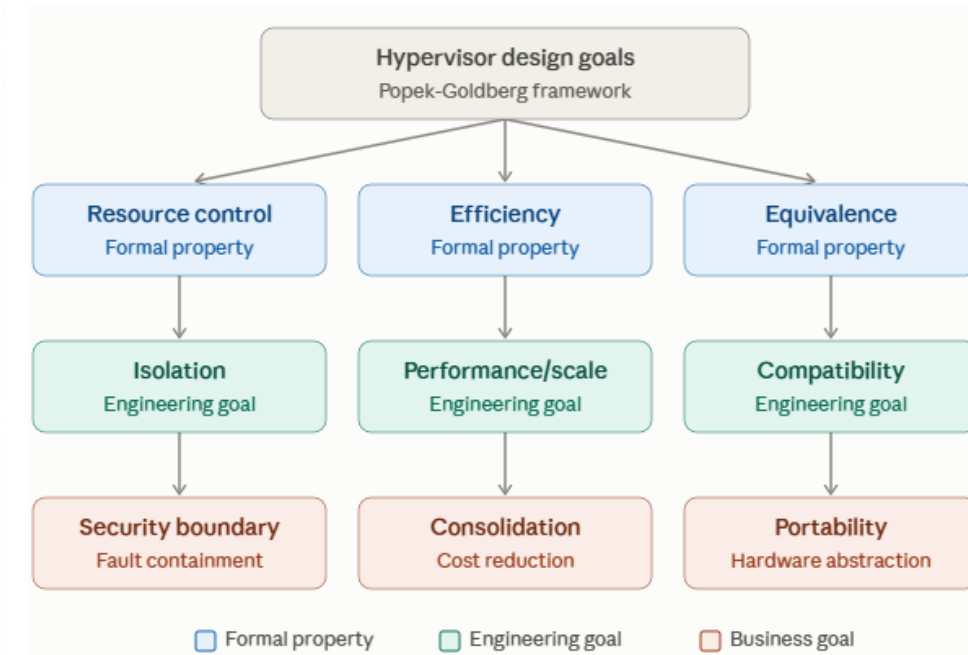
Bare Metal Hypervisor



Hosted Hypervisor

Hypervisor Goals

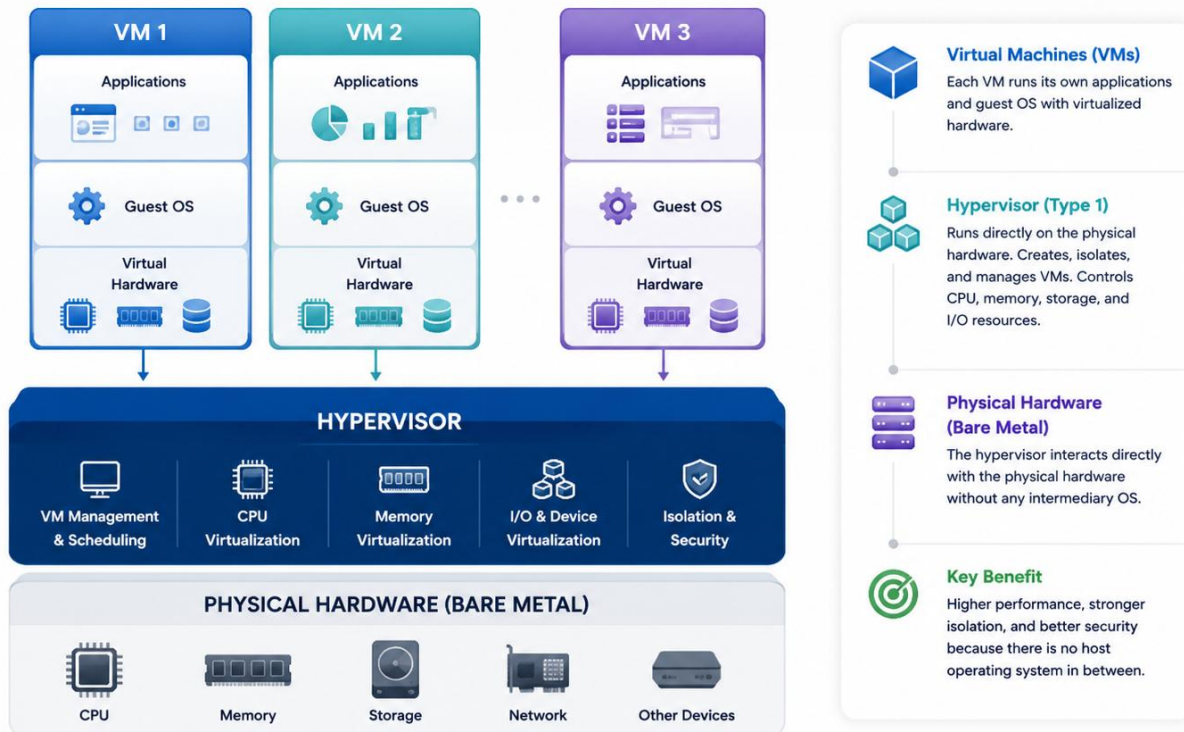
Category	Goal	What it means	Why it matters
 Formal property (Popek & Goldberg, 1974)	 Equivalence / Fidelity	Guest software behaves identically to running on bare hardware — no awareness of virtualization	Guest OS/apps need zero modification to run under the VMM
 Formal property (Popek & Goldberg, 1974)	 Resource control	VMM retains complete control over physical resources; guest cannot bypass it	Prevents one guest from touching hardware it shouldn't — foundation of isolation
 Formal property (Popek & Goldberg, 1974)	 Efficiency	Statistically dominant fraction of instructions execute directly on hardware, not trapped	Without this, virtualization is correct but too slow to be useful (pure software emulation)
 Practical / business goal	 Timesharing / resource multiplexing	Multiple users/workloads share one expensive physical machine	Origin motivation — 1960s mainframes were too costly to dedicate per user [Likely]
 Practical / business goal	 Isolation / fault containment	A crash in one VM doesn't affect others on the same host	Reliability at scale — one bad tenant shouldn't take down the box
 Practical / business goal	 Security boundary	Contain malicious or untrusted code within its VM	Multi-tenant cloud is only viable if tenants can't reach each other
 Practical / business goal	 Hardware abstraction / portability	Decouple guest OS from specific physical hardware	Same VM image runs on different underlying servers
 Practical / business goal	 Consolidation / cost reduction	Run many underutilized OS instances on fewer physical machines	Direct driver of x86 hypervisor adoption in early 2000s (VMware)
 Practical / business goal	 Operational agility	Enables snapshotting, live migration, fast provisioning	Second-order benefit — emerged after the core problem was solved, not an original design goal



Hypervisor Types

BARE METAL HYPERVISOR ARCHITECTURE (TYPE 1)

The hypervisor runs directly on physical hardware to create and manage virtual machines.



Examples of Type 1 Hypervisors

vmware

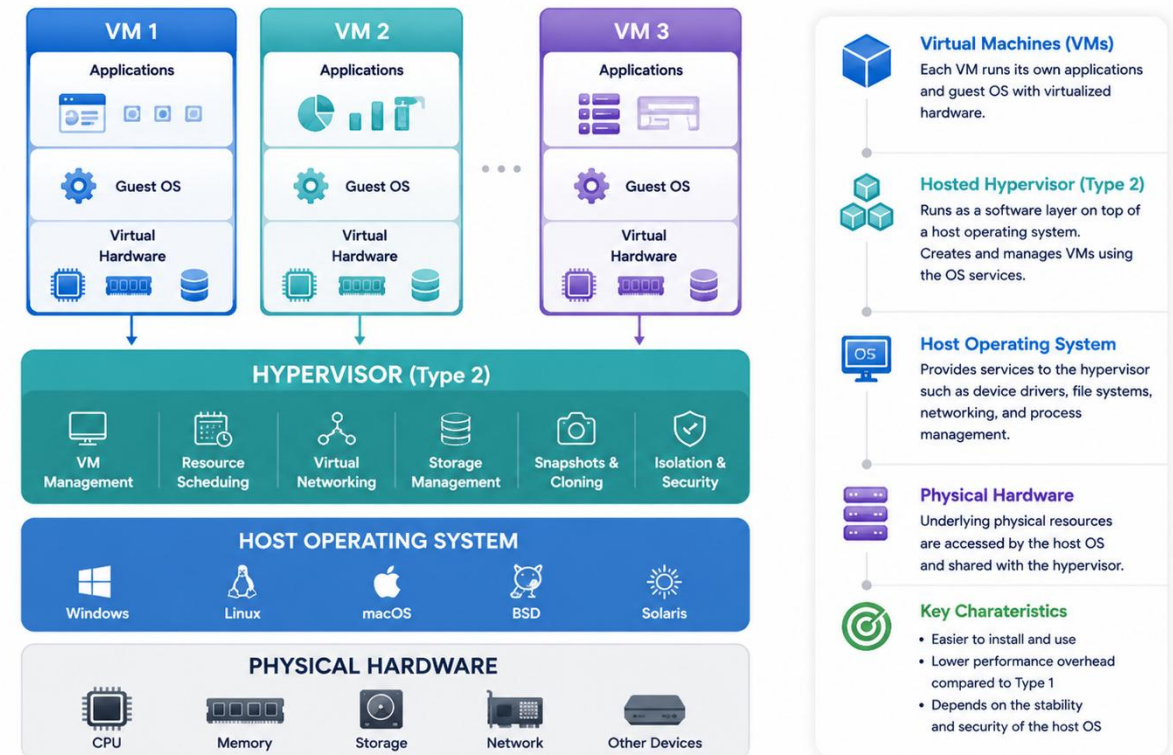
Microsoft Hyper-V

citrix Hypervisor

KVM

HOSTED HYPERVISOR ARCHITECTURE (TYPE 2)

The hypervisor runs as an application on a host operating system to create and manage virtual machines.



Examples of Type 2 Hypervisors

vmware Workstation Player / Pro

ORACLE VirtualBox

Parallels Desktop

citrix Workspace

Microsoft Virtual PC

Hypervisor Architecture

 Kernel type	 Core idea	 Where services run	 Defining mechanism	 Representative examples
 Microkernel	Kernel provides only the bare minimum needed to manage hardware; everything else is a replaceable, isolated component.	• Kernel space: IPC, minimal scheduling, minimal memory management only. • Drivers, file systems, networking: user space, as separate processes.	 Communication between kernel and services happens via message-passing (IPC) across a protected boundary.	 QNX, MINIX, GNU Hurd (Mach-based), L4 family (Fiasco, seL4, OKL4), Symbian (EKA2), INTEGRITY, PikeOS
 Monolithic kernel	Kernel is one large program containing all OS services, compiled and running as a single unit.	• Kernel space: everything — core, drivers, file systems, networking, all of it.	 Services call each other directly as function calls within one address space — no boundary to cross.	 Linux, the BSDs (FreeBSD/OpenBSD/NetBSD), classic Windows 9x (95/98/Me), Solaris, AIX, HP-UX, OpenVMS
 Hybrid kernel	Not a third clean architecture — a monolithic-leaning design that borrows the organizational modularity of microkernels without adopting their runtime isolation.	• Kernel space: most subsystems, same as monolithic. The “hybrid” part is that these subsystems are structured as internally modular, loadable components rather than one undifferentiated block.	 Retains direct in-kernel function calls (monolithic-style) for most services, with IPC used only for the smaller set of components actually pushed to user space.	 Windows NT-based (XP through 11), macOS/iOS (XNU), BeOS/Haiku

Comparison

Comparison of Implementation Levels

Level of Implementation	Higher Performance	Application Flexibility	Implementation Complexity	Application Isolation
ISA	X	XXXXX	XXX	XXX
Hardware-level virtualization	XXXXX	XXX	XXXXX	XXXX
OS-level virtualization	XXXXX	XX	XXX	XX
Runtime library support	XXX	XX	XX	XX
User application level	XX	XX	XXXXX	XXXXX

Scale: X (Lowest) XX XXX XXXX XXXXX (Highest)

The number of X's in the table cells reflects the advantage points of each implementation level. Five X's implies the best case and one X implies the worst case.

Overall, hardware and OS virtualization will yield the highest performance. However, the hardware and application levels are also the most expensive to implement. User isolation is the most difficult to achieve. ISA implementation offers the best application flexibility.

Application level

JVM / .NET CLR / Panot

Library (user-level API) level

WINE/ WABI/ LxRun / Visual MainWin / vCUDA

Operating system level

Jail / Virtual Environment / Ensim's VPS / FVM

Hardware abstraction layer (HAL) level

VMware / Virtual PC / Denali / Xen / L4 /
Plex 86 / User mode Linux / Cooperative Linux

Instruction set architecture (ISA) level

Bochs / Crusoe / QEMU / BIRD / Dynamo

Video – BOCHS



Learning Objectives

- Hypervisors can be used at any **abstraction level**.
- Oracle Virtual Box was an example of an **hardware abstraction**.
- We will see Bochs (pronounced Box), which is an **ISA abstraction**.
- It enables emulation to disparate Instruction sets



innovate

achieve

lead

BITS Pilani

Pilani|Dubai|Coa|Hyderabad

IAAS

Infrastructure as a Service - Recap

innovate

achieve

lead

Why IaaS



Capability

IaaS provides the following

- Servers- compute, machines
- Storage
- Network
- Operating system



IaaS delivers the foundational building blocks to run IT workloads in the cloud.



Models

IaaS can be obtained as

- 1 Public or
- 2 Private infrastructure or
- 3 combination of both



Choose the right model for your business needs and compliance requirements.



IaaS

Infrastructure as a Service

Characteristics



Resources are distributed as a service

- Allows dynamic scaling (1...10....100....)
- Has a variable costs-
- Generally includes multiple-users on a single piece of hardware. (**multi-tenancy**)



Scale up or down on demand and pay only for what you use.

Benefit



The user instead of **purchasing**-servers, software, **data center space** or network equipment, **rent** those resources as a fully **outsourced** service on-demand model.



Reduce CapEx. Increase agility. Focus on innovation, not infrastructure.

Enabler : Virtualization Technology



Manageability and Interoperability

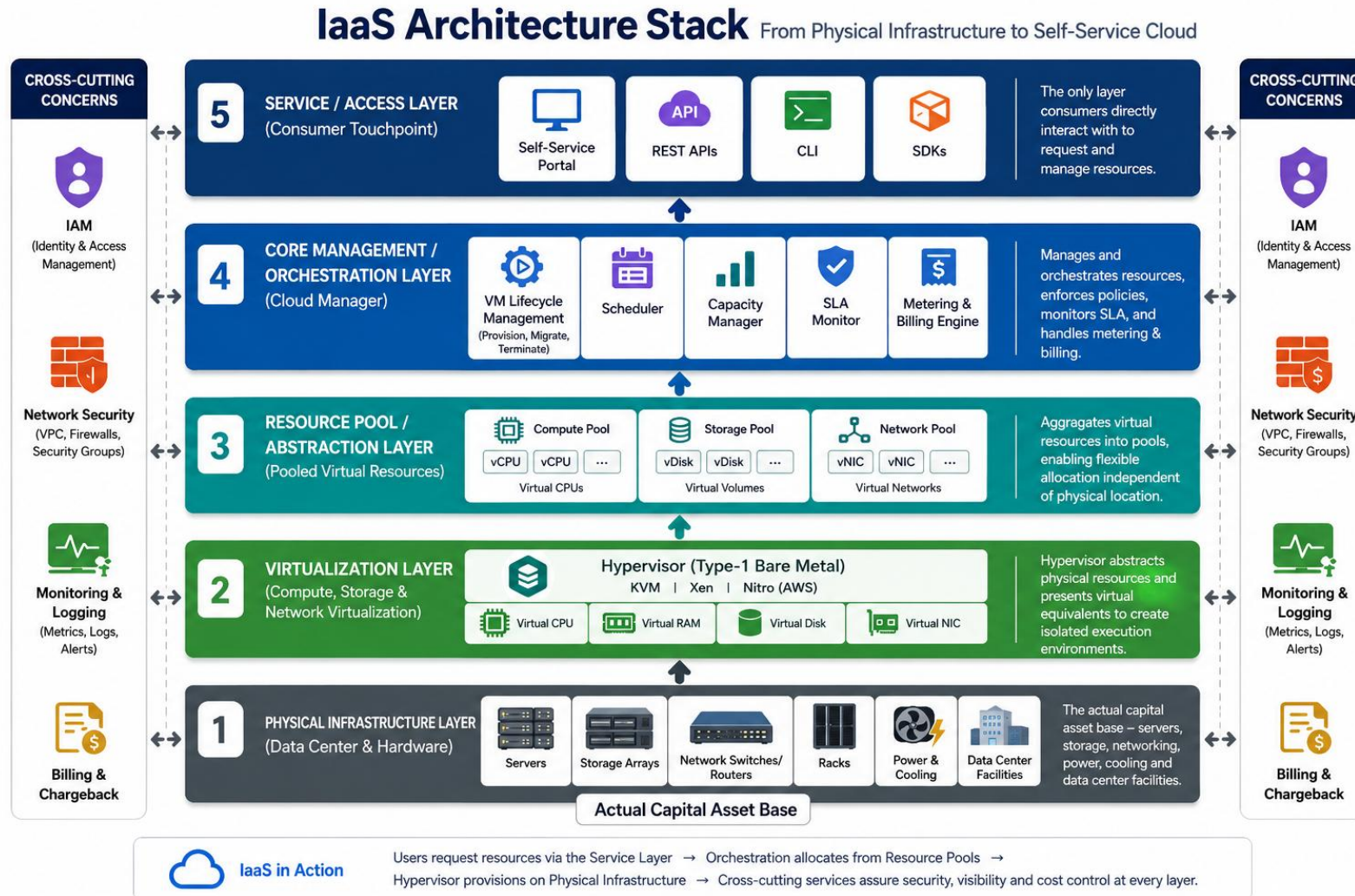


Availability and Reliability



Scalability and Elasticity

IAAS Architecture Model



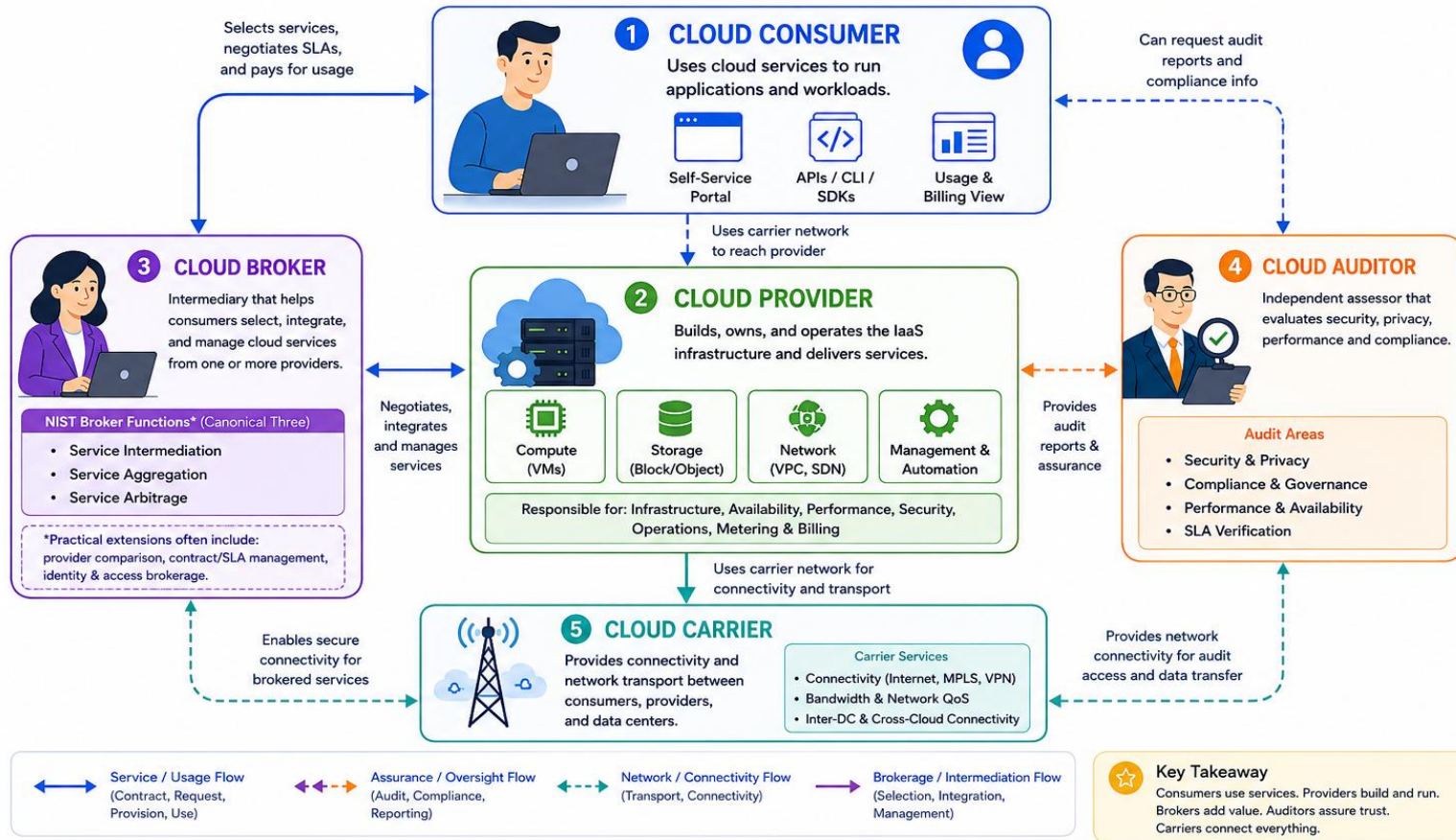
This figure shows how the various technologies we learnt come together.

The final outcome of the interactions is what we colloquially refer to IaaS

IAAS Actors

IaaS Actors and How They Interact

Roles in the Cloud Business/Interaction Model (NIST SP 500-292)



Note: Provider sub-services (Compute, Storage, Network, Management) are IaaS-specific specializations used for clarity in this diagram. NIST defines the Provider as a single actor; these sub-services are not part of the canonical NIST actor model.

Key Takeaway
Consumers use services. Providers build and run. Brokers add value. Auditors assure trust. Carriers connect everything.

Cloud Consumer — Uses services

Cloud Provider — Builds & operates

Cloud Broker — Intermediates/adds value

Cloud Auditor — Assesses/verifies

Cloud Carrier — Connects/transport



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Introducing Amazon Web Service



AWS Reference Architecture – Complete Service Landscape (2025)

Build, Modernize and Innovate with the Broadest and Deepest Cloud Services

APPLICATION SERVICES

Application Integration



Developer Tools



User Engagement



Frontend & Mobile



End User Computing



Media Services



DATA & ANALYTICS

Data Ingestion



Analytics & Processing



Data Warehousing & Lake



Databases



Data Migration & Transfer



AI, ML & GENERATIVE AI

AI/ML Services



Foundation Models & Generative AI



ML Operations



Search & Conversational AI



FOUNDATION SERVICES

Compute



Storage



Networking & Content Delivery



Security, Identity & Compliance



Observability & Management



Enterprise Integration



Hybrid & Edge



Containers & Orchestration



MANAGEMENT & GOVERNANCE

Governance



Account Management



Monitoring & Audit



Automation



Cost Management



Marketplace & Ecosystem



GLOBAL INFRASTRUCTURE



Regions

Multiple AWS Regions around the world



Availability Zones

Isolated locations within an AWS Region



Local Zones

Extensions of Regions closer to population



Wavelength Zones

Ultra-low latency 5G edge compute



AWS Edge Locations

CloudFront Edge Locations (200+ Points of Presence)



Broadest Services. Deepest Capabilities.
From Infrastructure to Applications, Data to AI – AWS has you covered.



Build



Run



Secure



Optimize



Connect



Analyze



AI/ML

What is AWS



Amazon Web Services (AWS) is a **Public Cloud Provider** offering over 240 fully **featured On-demand, pay-as-you-go services** from data centers globally. Millions of customers—including the fastest-growing startups, largest enterprises, and leading government agencies—are using AWS to lower costs, become more agile, and innovate faster.



Global Infrastructure: AWS serves over one million active customers in more than 190 countries, and it continues to expand its global infrastructure. **Virtualized compute, storage, network, and platform-level services** are grouped and made available by **Regions and availability zones**.

Security: All AWS customers benefit from data center and network architectures built to satisfy the requirements of the most security-sensitive organizations.



- **Self-service API/console/CLI layer**
- Backed by Regions & AZ redundancy
- Shared responsibility – Security model

AWS Understanding Service Offering

- AWS operates **state-of-the-art, highly available data centers**. Although rare, failures can occur that affect the availability of instances that are in the same location.
- If you **host all of your instances** in a **single location** that is affected by a failure, none of your instances would be available.
- Amazon EC2 is hosted in multiple locations world-wide. These locations are composed of **AWS Regions**, **Availability Zones**, **Local Zones**, **AWS Outposts**, and **Wavelength Zones**.



Availability Zones located in **AWS Regions** consist of one or more **discrete data centers**, each of which has **redundant power**, **networking**, and **connectivity**, and is housed in **separate facilities**. Each AZ has multiple internet connections and power connections to multiple grids.

AWS Understanding Service Offering

AWS Global Infrastructure – The Hotel Chain Analogy

AWS is designed for high availability, resiliency and low latency by distributing infrastructure at multiple levels.



Region — a geographic area (e.g., Mumbai, Frankfurt).

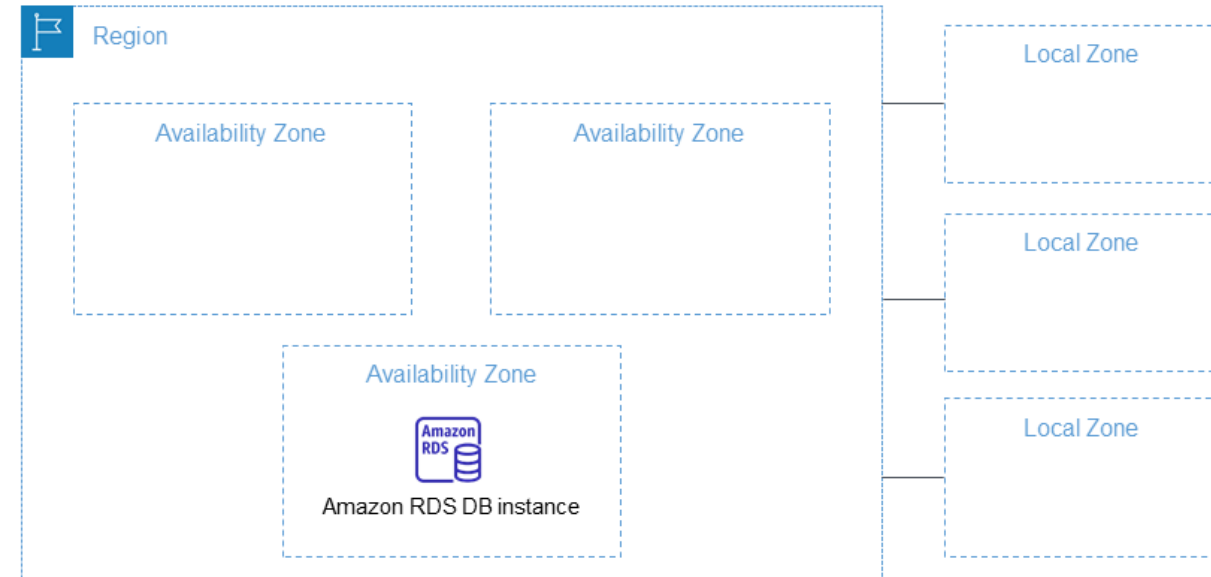
Not a building, not even a single site — it's a naming boundary that groups a cluster of nearby AZs together.

Availability Zone — one or more discrete data centers, in a specific location within that region, that share a dedicated power substation, cooling, and network uplinks, and are physically separated from the other AZs in the same region (different flood plain, different power grid, enough distance that one disaster doesn't take out two AZs at once).

Data Center — the actual physical building: the racks, the servers, the concrete and cabling. This is the only one of the three terms with a real street address.

AWS Regions

- AWS provides a highly available technology infrastructure platform with multiple **locations worldwide**. These locations are composed of regions and **Availability Zones**. Each **region** is a **separate geographic area**.
- It is important to remember that each AWS Region is completely independent. Any Amazon service you initiate (for example, creating database instances or listing available database instances) runs only in your current default AWS Region.
- The default AWS Region can be changed in the console, or by setting the AWS_DEFAULT_REGION environment variable. Or it can be overridden by using the --region parameter with the AWS Command Line Interface (AWS CLI).
- Each AWS Region is designed to be isolated from the other AWS Regions. This design achieves the greatest possible fault tolerance and stability.

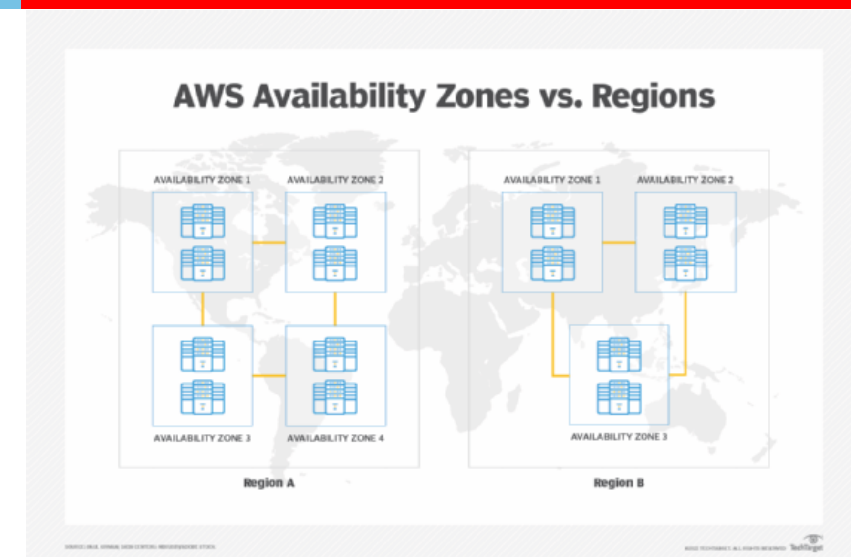


Availability Zones located in **AWS Regions** consist of one or more **discrete data centers**, each of which has **redundant power**, **networking**, and **connectivity**, and is housed in **separate facilities**.

Each AZ has multiple internet connections and power connections to multiple grids.

AWS Availability Zones

- Each Region has multiple, isolated locations known as **Availability Zones**. Each **Availability Zone** is also **isolated**, but the **Availability Zones** in a **region** are connected through **low-latency links**.
- **Availability Zones** are **physically separated** within a typical metropolitan region and are located in lower-risk flood plains (specific flood zone categorization varies by region). In addition to using a **discrete uninterruptible power supply (UPS)** and on-site backup generators, they are each fed via **different grids** from **independent utilities** (when available) to reduce single points of failure further.
- **Availability Zones** are all **redundantly connected** to multiple tier-1 transit providers. By placing resources in **separate Availability Zones**, you can protect your website or application from a **service disruption** impacting a single location.
- The code for Availability Zone is its Region code followed by a letter identifier. For example, us-east-1a.



- If you **distribute** your instances across **multiple Availability Zones** and one instance fails, you can design your application so that an instance in another Availability Zone can handle requests.

AWS Local Zones

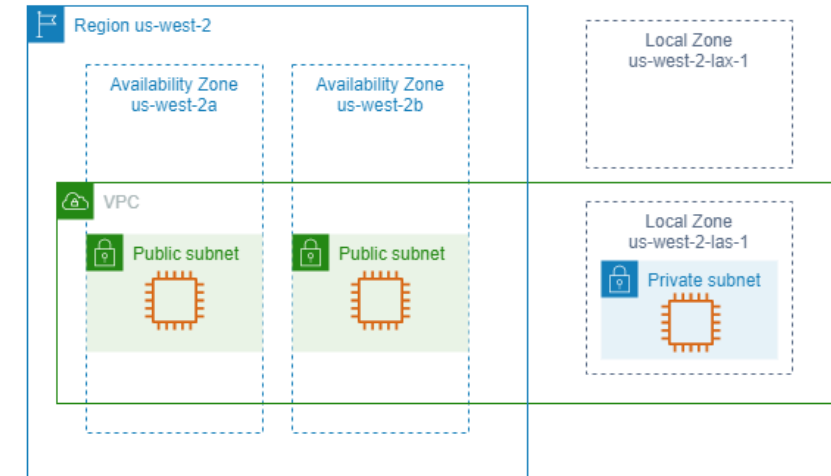
A **Local Zone** is an extension of an **AWS Region** in geographic proximity to your users.

Local Zones have their own connections to the internet and support AWS Direct Connect, so that resources created in a Local Zone can serve local users with low-latency communications.

The code for a Local Zone is its Region code followed by an identifier that indicates its physical location. For example, `us-west-2-lax-1` in Los Angeles.

The **VPC** spans the **Availability Zones** and one of the Local Zones. Each **zone** in the **VPC** has one **subnet**, and each **subnet has an instance**.

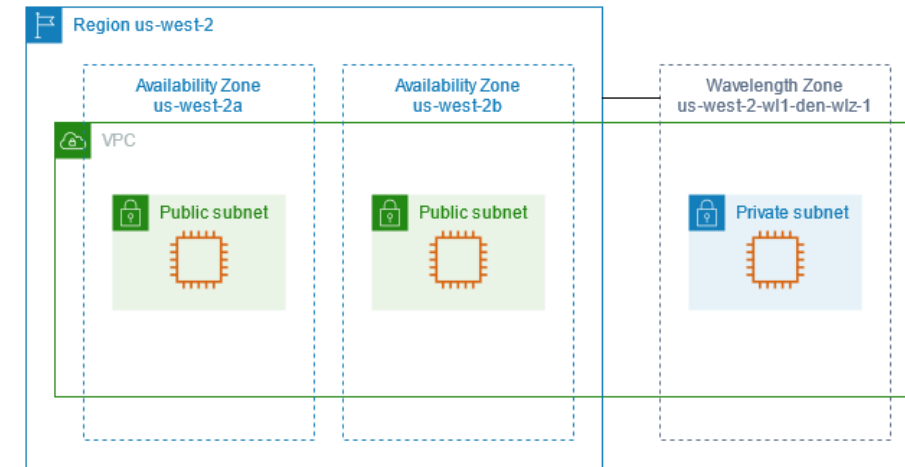
When you **launch an instance**, you can specify a **subnet that is in a Local Zone**. You also allocate an IP address from a network border group. A network border group is a unique set of Availability Zones, Local Zones, or Wavelength Zones from which AWS advertises IP addresses, for example, `us-west-2-lax-1a`.



- Some **AWS resources** might not be available in all **Regions**. Make sure that you can create the resources that you need in the desired Regions or Local Zones before launching an instance in a specific Local Zone
- Before you can **specify a Local Zone** for a **resource** or service, you must **opt in** to Local Zones.

AWS Wavelength Zones

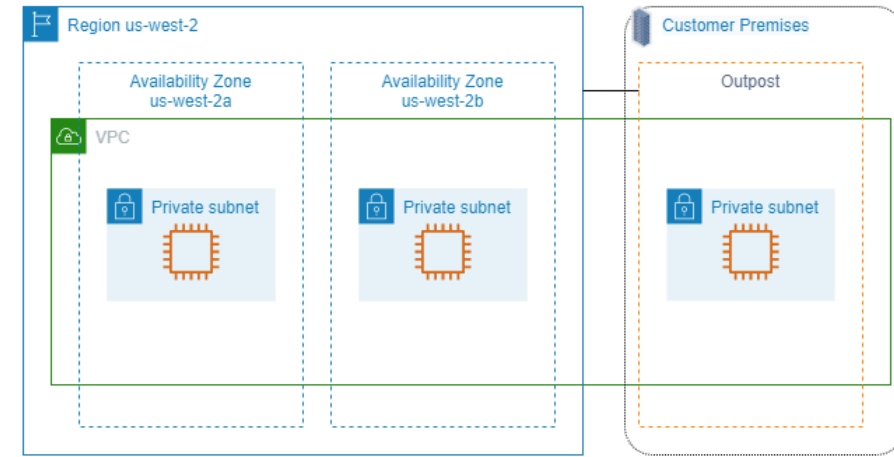
- **Wavelength Zones** are **AWS infrastructure deployments** that embed AWS compute and storage services within **telecommunications providers' data centers** at the edge of the **5G network**, so application traffic can reach application servers running in Wavelength Zones without leaving the mobile providers' network.
- This **prevents the latency** that would result from multiple hops to the internet and enables customers to take **full advantage of 5G networks**. Wavelength Zones extend **AWS to the 5G edge**, delivering a consistent developer experience across **multiple 5G networks around the world**. Wavelength Zones also allow developers to **build** the next generation of **ultra-low latency applications** using the same **familiar AWS services**, APIs, tools, and functionality they already use today.
- Processing at the network edge can help avoid transmitting large volumes of data over the network provider's infrastructure, and offload processing from mobile device hardware.



This enables new classes of compute-intensive, latency sensitive applications. For example, a **fleet of autonomous cars** interacting with road sensors to prevent crashes, smart industrial robots assessing and reacting to plant conditions in a dangerous manufacturing environment, or retailers serving **personalized promotions** to shoppers' **mobile phones** in real time as they pass **product displays**.

AWS Outposts

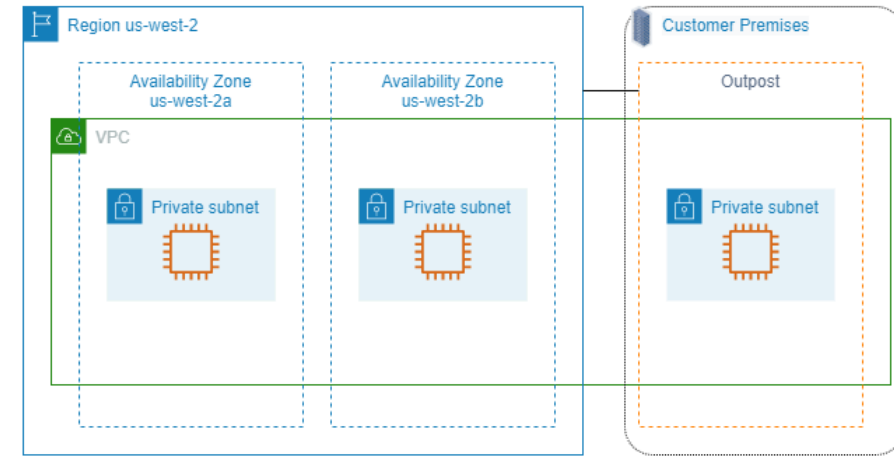
- **AWS Outposts** is a fully **managed service** that extends **AWS infrastructure**, services, APIs, and tools to **customer premises**. By providing **local access** to **AWS managed infrastructure**, AWS Outposts enables customers to build and run **applications on premises using the same programming interfaces as in AWS Regions**, while **using local compute and storage resources for lower latency** and local data processing needs.
- **AWS operates, monitors, and manages this capacity** as part of an **AWS Region**. You can create subnets on your Outpost and specify them when you create AWS resources. Instances in Outpost subnets communicate with other instances in the AWS Region using private IP addresses, all within the same VPC.
- The following diagram illustrates the AWS Region us-west-2, two of its Availability Zones, and an Outpost. The VPC spans the Availability Zones and the Outpost. The Outpost is in an on-premises customer data center. Each zone in the VPC has one subnet, and each subnet has an instance.



- This enables new classes of compute-intensive, latency sensitive applications. For example, a bank running core transaction processing on-prem for data residency while still using AWS APIs, or a factory floor needing local processing where connectivity to AWS region is unreliable.

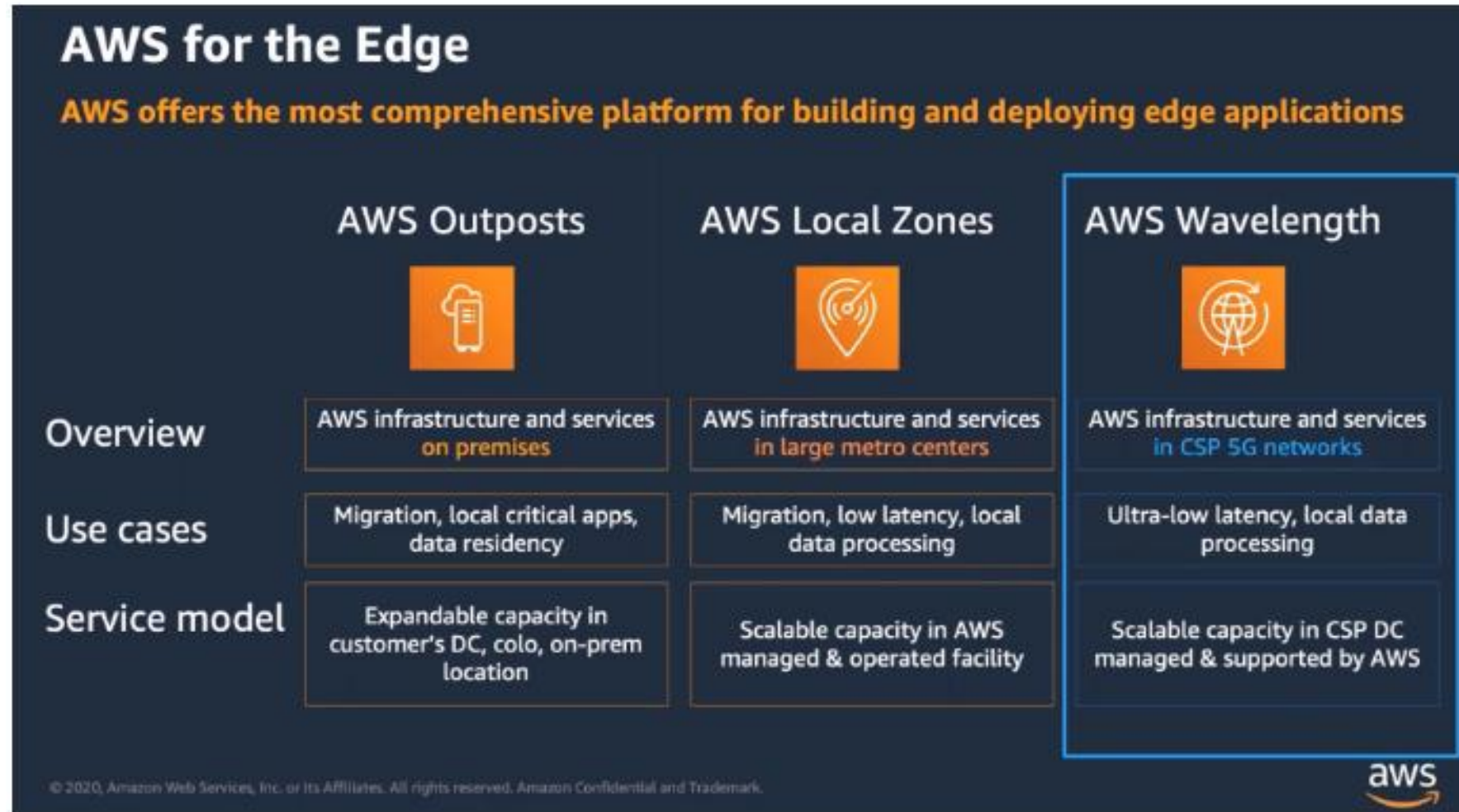
AWS Outposts

- **AWS Outposts** is a fully **managed service** that extends **AWS infrastructure**, services, APIs, and tools to **customer premises**. By providing **local access** to **AWS managed infrastructure**, AWS Outposts enables customers to build and run **applications on premises using the same programming interfaces as in AWS Regions**, while **using local compute and storage resources for lower latency** and local data processing needs.
- **AWS operates, monitors, and manages this capacity** as part of an **AWS Region**. You can create subnets on your Outpost and specify them when you create AWS resources. Instances in Outpost subnets communicate with other instances in the AWS Region using private IP addresses, all within the same VPC.
- The following diagram illustrates the AWS Region us-west-2, two of its Availability Zones, and an Outpost. The VPC spans the Availability Zones and the Outpost. The Outpost is in an on-premises customer data center. Each zone in the VPC has one subnet, and each subnet has an instance.



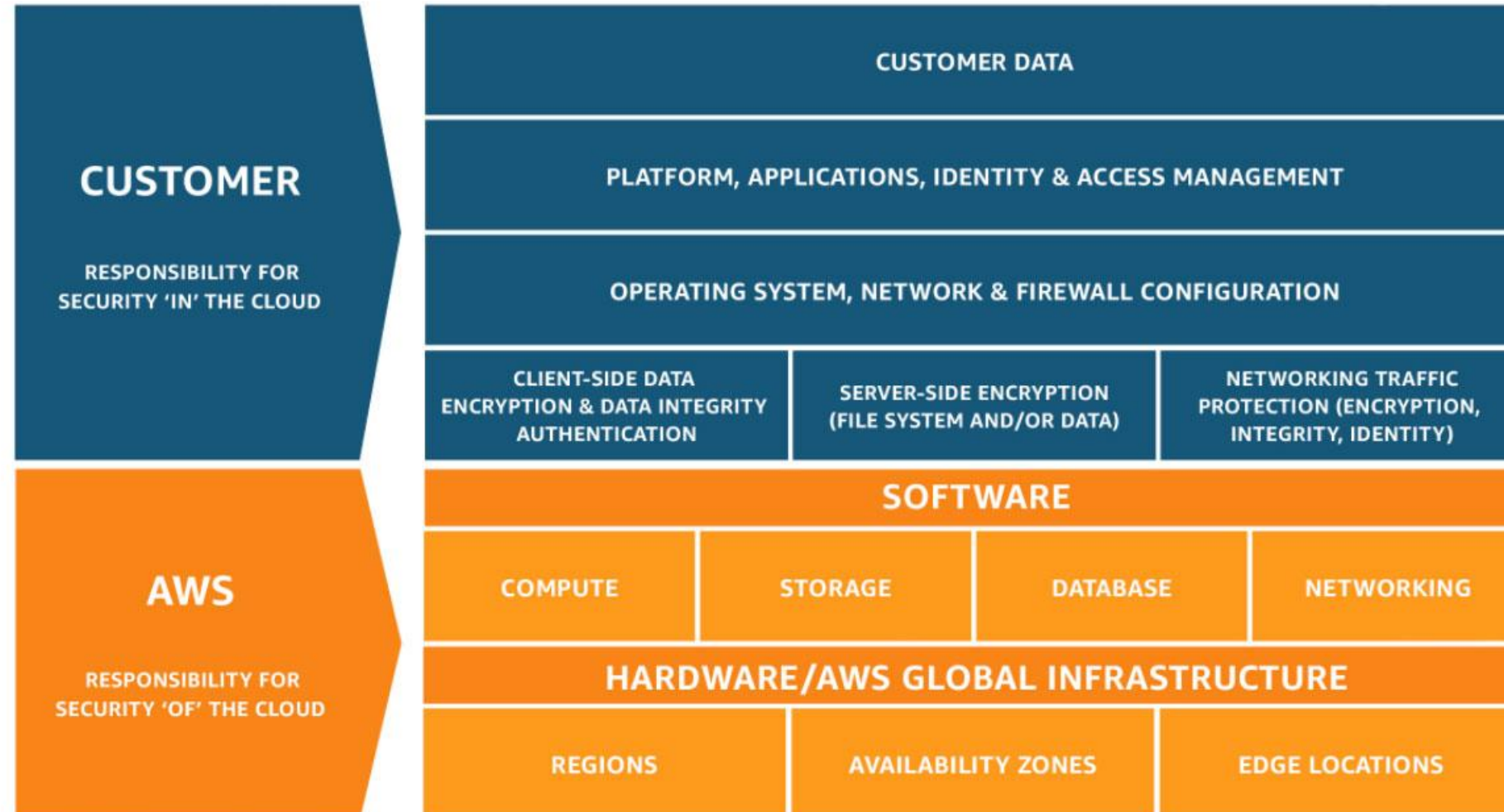
- This enables new classes of compute-intensive, latency sensitive applications. For example, a bank running core transaction processing on-prem for data residency while still using AWS APIs, or a factory floor needing local processing where connectivity to AWS region is unreliable.

AWS for the Edge



AWS edge computing infrastructure and service platforms. Source: AWS

AWS & Customer



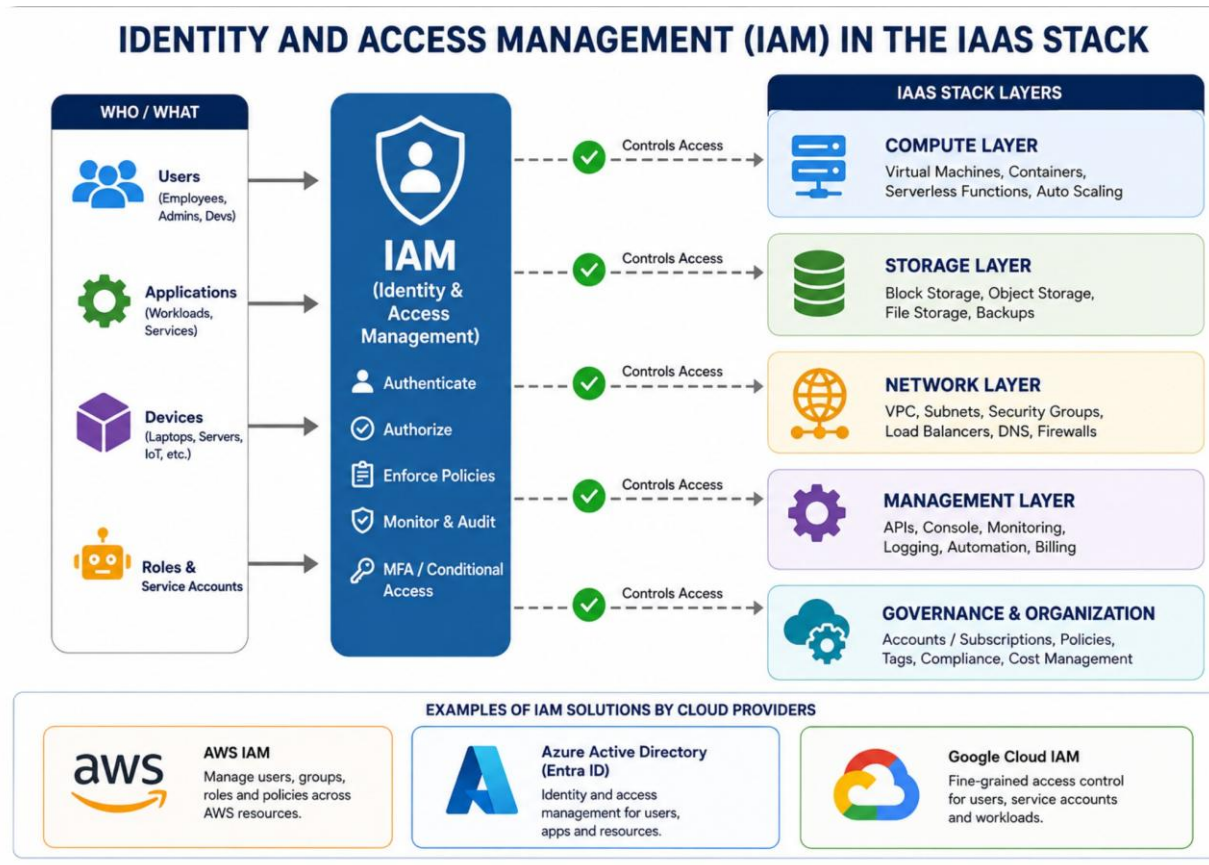


BITS Pilani

Pilani | Dubai | Goa | Hyderabad

IAM, Authentication & Authorization, Roles & Policies...

Identity Access Management – IAM - Overview



What

IAM manages digital identities and controls resource access. It's the layer that decides who (or what) can access which resource, and under what conditions.

Why

In IaaS, every resource is network-reachable by default — there's no physical wall. IAM replaces the locked door. Without it, questions like "who can spin up a \$50K/month cluster?" have no answer.

How

IAM is a cross-cutting layer gating access across compute, storage, network, and management simultaneously — not siloed in one place.

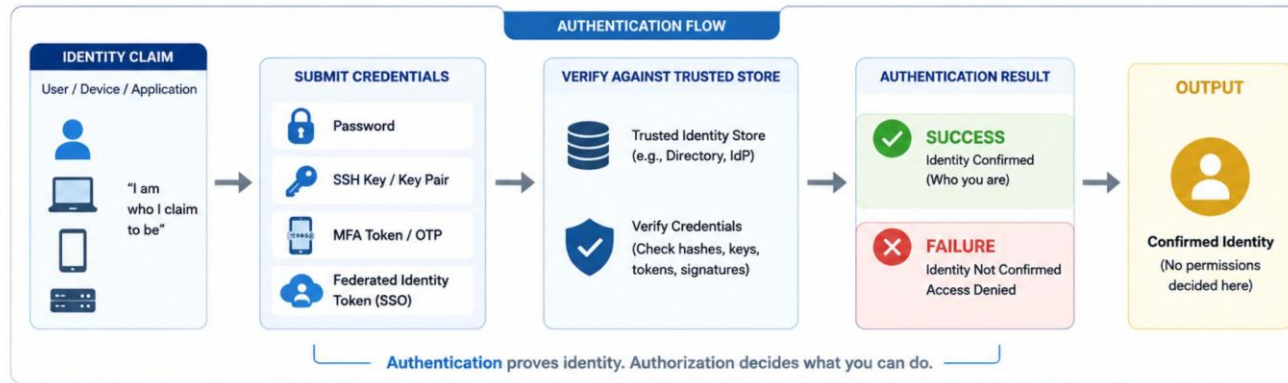
Examples

AWS IAM, Azure Active Directory (Entra ID), Google Cloud IAM — every major provider requires one.

Authentication: "Who are you?"

AUTHENTICATION: VERIFYING IDENTITY

Proving who you are — not what you can do



EXAMPLE AUTHENTICATION SCENARIOS

1 User Login to AWS Console (with username/password + MFA)



2 EC2 Instance authenticates to another AWS Service



3 SSH Key-Pair Login to Linux VM



KEY TAKEAWAY

Authentication answers the question "Who are you?" by verifying identity through trusted methods. It does not decide what you can access — that's the job of Authorization.



Something you know (password)

Common Authentication Factors



Something you have (token, phone)



Something you are (biometric)



Something you have (key pair)

What

Authentication verifies that an identity is who it claims to be — proving identity, not granting access.

Why

A system must know who's asking before deciding what they can do. Without it, anyone claiming to be "admin" gets treated as one — the entire access model collapses.

How

Credentials are checked against a trusted store: password + hash, cryptographic key pairs, MFA tokens, or federated identity (SSO). The output is a confirmed identity only — what that identity can *do* is determined separately (authorization).

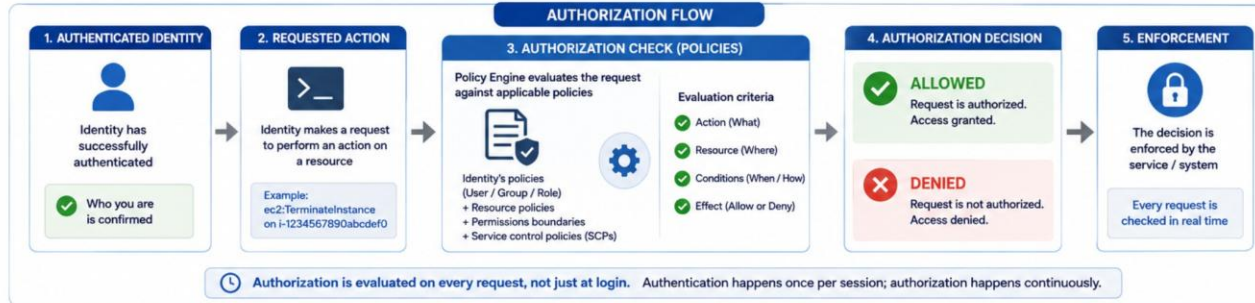
Examples

AWS Console login with username/password + MFA; EC2 instance authenticating to AWS services via temporary credentials; SSH key-pair login to a Linux VM.

Authorization: "What are you allowed to do?"

AUTHORIZATION: WHAT ARE YOU ALLOWED TO DO?

Determining what an authenticated identity is permitted to do



What

Authorization determines what an already-authenticated identity is permitted to do — which actions, on which resources.

Why

Authentication alone isn't access control. Without authorization, every logged-in user could delete the production database. It enforces least privilege — each identity gets only the access it actually needs.

How

On every request, the system checks the authenticated identity against policies that explicitly allow or deny specific actions on specific resources. Unlike authentication (once per session), authorization is evaluated continuously.

Examples

An AWS user gets "Access Denied" trying to terminate an EC2 instance if their policy lacks `ec2:TerminateInstance` — even after successful login. A read-only analyst can SELECT from a database but is blocked on DELETE.

Roles: Identity Without a Person Attached

ROLES: IDENTITY WITHOUT A PERSON ATTACHED

Temporary, least-privilege access for anyone or anything that needs it

WHAT IS A ROLE?



A role is a set of permissions that can be assumed temporarily by a trusted identity — a human, application, or AWS service.



Unlike a user, a role has no long-term password or access key of its own.



It exists only for the purpose of being assumed to get temporary access.

WHY USE ROLES?



ELIMINATES LONG-TERM CREDENTIALS
No permanent keys stored in code, on servers, or in configuration files.



REDUCES RISK
Short-lived, automatically-rotated credentials limit the blast radius if compromised.



ENABLES SECURE AUTOMATION
Services and applications get the identity they need to act — without a person's credentials.



ENFORCES LEAST PRIVILEGE
Permissions are scoped to exactly what the role needs — nothing more.

HOW ROLES WORK



EXAMPLES OF ROLES IN ACTION

1 EC2 Instance Role – Read from S3



The EC2 instance gets temporary credentials to read from a specific S3 bucket. No AWS keys are stored on the instance.

2 Lambda Execution Role – Write to DynamoDB



The Lambda function gets temporary credentials to write to a specific DynamoDB table and nothing else.

3 Cross-Account Role – Auditor Access



The auditor assumes a role in your account to inspect logs without needing any permanent credentials to your account.



KEY TAKEAWAY

Roles provide temporary, least-privilege access without permanent credentials, solving both security risks and automation identity needs.

ROLE vs. USER

ROLE	VS.	USER
No long-term password or access key		Has long-term password or access keys
Assumed by trusted identities (people, apps, services)		Represents a human identity
Temporary credentials (minutes to hours)		Credentials are long-lived (unless rotated manually)
Best for applications, services, automation, cross-account access		Best for human users who log in
Scoped to least-privilege permissions		Often broader access than needed

What

A role is a set of permissions that can be assumed — temporarily and without permanent credentials — by a human, application, or AWS service. Unlike a user, a role has no long-term password or access key.

Why

Hardcoding permanent credentials is a critical security anti-pattern — if compromised, attackers get permanent, exportable keys. Roles issue short-lived, auto-rotating credentials scoped to exactly what's needed. They also solve the "who acts on whose behalf" problem in automation (e.g., a Lambda function needs an identity but can't type a password).

How

An identity "assumes" a role and receives temporary credentials (valid minutes to hours) granting only that role's permissions — then they expire automatically. No permanent keys stored anywhere.

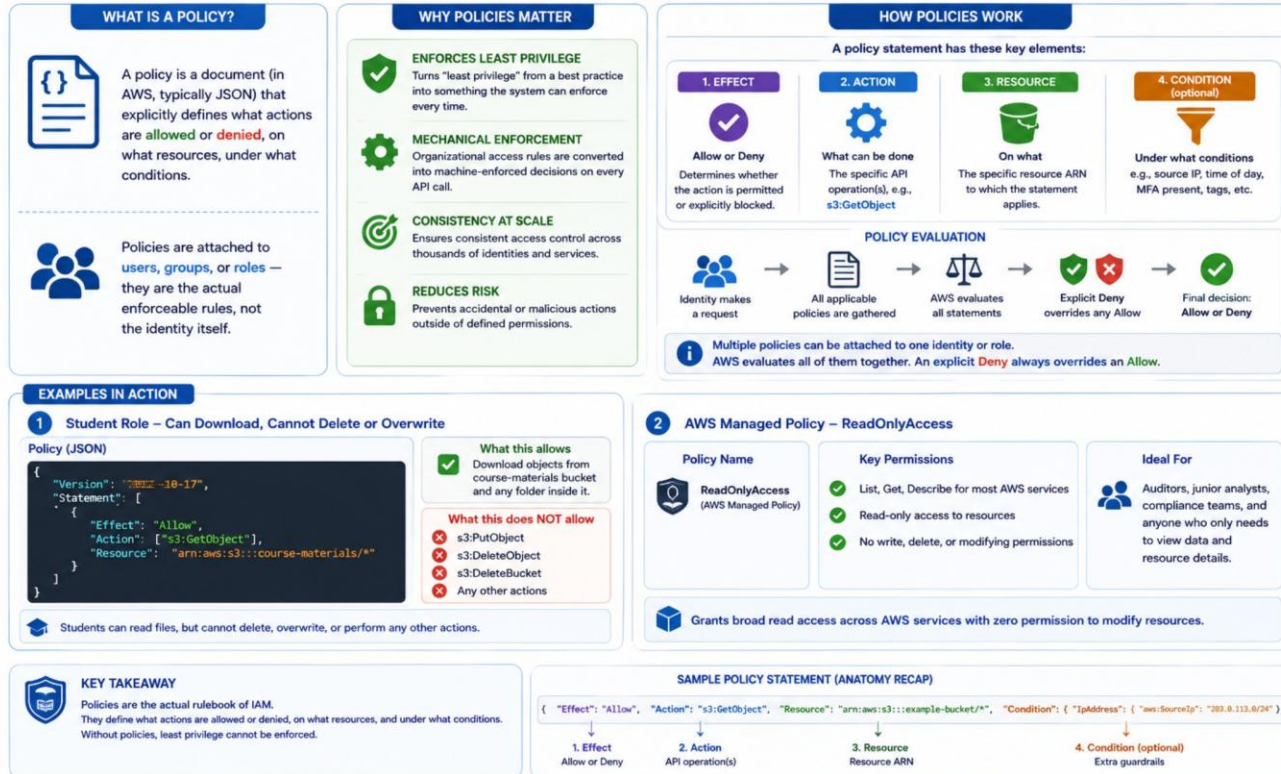
Examples

EC2 instance reading from a specific S3 bucket via an IAM role — no AWS keys on the instance. Lambda function writing to a specific DynamoDB table via its execution role. A third-party auditor assuming a cross-account role to inspect logs without holding a permanent password.

Policies: Rule Book

POLICIES: THE ACTUAL RULEBOOK

The rules that specify what's allowed or denied, on what, and under what conditions



What

A policy is a document (typically JSON in AWS) that explicitly defines what actions are allowed or denied, on what resources, under what conditions. Policies attach to users, groups, or roles — they are the actual enforceable rules, not the identity itself.

Why

Without machine-enforced policies, "least privilege" is just a phrase. Policies convert organizational rules ("interns can't delete anything") into something the system checks mechanically on every single API call — at any scale.

How

Each policy statement specifies: **Effect** (Allow/Deny), **Action** (e.g. `s3:GetObject`), **Resource** (specific bucket/instance), and optionally **Condition** (e.g. specific IP or time of day). Multiple policies can attach to one identity; an explicit Deny always overrides an Allow.

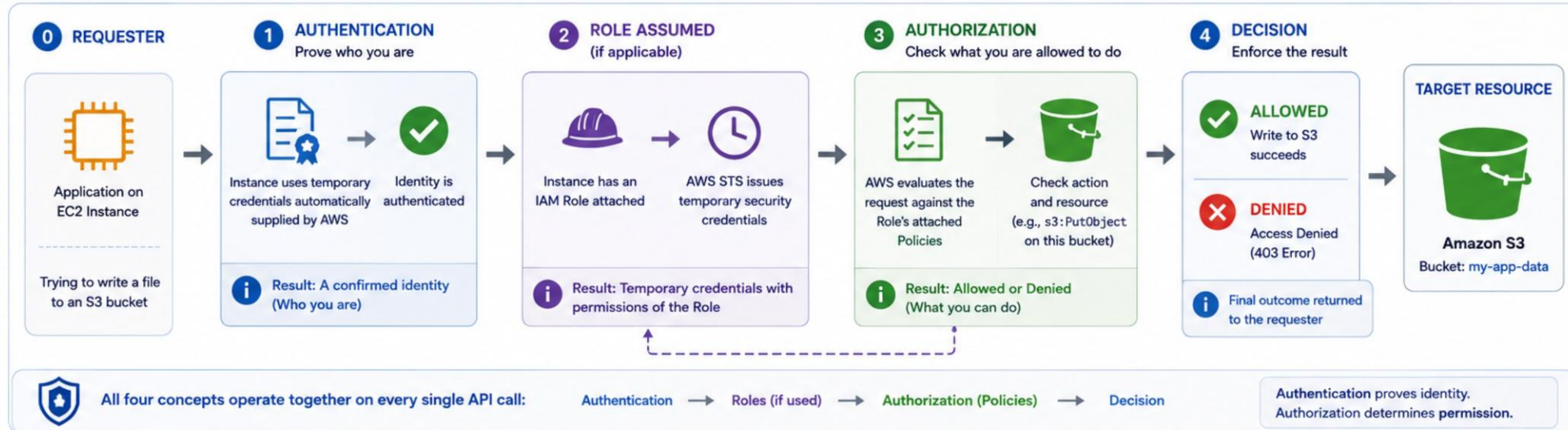
Examples

A student role policy granting `s3:GetObject` on `course-materials/*` only — can download, cannot delete or overwrite. AWS managed policy `ReadOnlyAccess` — broad read permission, zero write/delete, commonly used for auditor and junior-analyst roles.

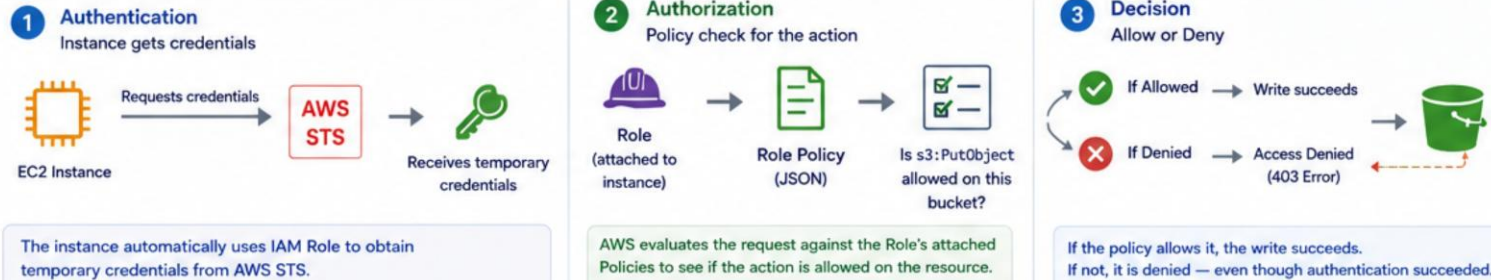
The IAM TANGO

PUTTING IT TOGETHER: THE IAM REQUEST FLOW

Every request goes through Authentication, then Authorization (via Policies), and Roles when used



WORKED EXAMPLE: EC2 INSTANCE WRITES TO S3



KEY POINTS

- ✓ Authentication answers: "Who are you?"
- ✓ Authorization answers: "What are you allowed to do?"
- ✓ Roles provide temporary, least-privilege credentials — no long-term keys.
- ✓ Policies are the rulebook that decides allow or deny.
- ✓ This entire flow happens on every single API call.

EXAMPLES OF THIS FLOW IN ACTION

- EC2 to S3**
Instance writes logs to a specific S3 bucket using its IAM Role.
- Lambda to DynamoDB**
Lambda function uses its execution role to write to a DynamoDB table.
- Cross-Account Audit**
External auditor assumes a role in your account to read logs (read-only).

LAB PREVIEW

- In the lab, you will:
1. Attach an IAM Role to an EC2 instance
 2. Create a policy that allows only the specific S3 actions needed (e.g., s3:PutObject on one bucket)
 3. Verify the instance can write to that bucket — and is denied for everything else
 4. Observe the full IAM request flow in action

REMEMBER

- Never give broad admin access "just to make it work."
- Use Roles + least-privilege Policies to give only the permissions needed, and nothing more.

Using AWS - Video

greatlearning
Power Ahead



AWS Management Console: is a web application for managing AWS Cloud services.

The console provides an intuitive user interface for performing many tasks.

Each service has its own console, which can be accessed from the AWS Management Console.

The console also provides information about the account and billing.

Q & A.....





BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Credits

*Hwang, Kai; Dongarra, Jack; Fox, Geoffrey C.. Distributed and Cloud Computing: From Parallel Processing to the Internet of Things (Kindle Locations 3532-3533). Elsevier Science. Kindle Edition.